Churn Prediction – Technical Report

Oct 20, 2026 · @Mine

1. Problem and use

I recommend a **Logistic Regression** model that ranks customers by their risk of churning within 30 days. On held-out test data it reaches ROC-AUC 0.76 and finds 2.1× more churners than random targeting.

Each row is one customer on the snapshot date 2026-08-01. The target `churned_30d` is 1 if the customer leaves within 30 days, so this is binary classification. The model would score all active customers monthly, so only information known on the snapshot date may be used. The retention team would contact customers from the top of the list; because capacity is limited, ranking matters more than a yes/no answer.

2. Data understanding

The data has 1,818 rows and 20 columns. 18% of customers churn, so accuracy is misleading: always predicting "stays" scores 82%. I removed 18 exact duplicates (1,800 rows left), set session time to missing for 79 customers with zero logins (an impossible combination), and noted that NPS has decimals (probably an average). Missing values in six features, all under 8%, are imputed in the pipeline.

Leakage. `days_until_churn` exists only for churners, so it reveals the target. `retention_offer_sent` (64.5% churn with an offer vs 8.4% without) is a reaction to known churn and the very decision the model should support. Both were dropped, together with the ID, the constant snapshot date and `signup_date` (it duplicates tenure).

3. Data preparation

An 80/20 stratified train/test split; the test set was used once, at the end. Imputation (median plus a missing indicator, or a "missing" category), scaling and one-hot encoding sit in a scikit-learn pipeline with the model, so in each of the 5 stratified CV folds they learn from the training folds only. Class imbalance is handled with `class_weight="balanced"` and a tuned threshold.

4. Baseline

Always predicting "stays" gives 82% accuracy but 0% recall. A business rule (monthly contract and fewer than 10 login days) reaches ROC-AUC 0.62 and PR-AUC 0.26: the bar a model must clear to be worth its complexity.

5. Models and experiments

All models used the same pipeline and the same 5 CV folds:

Model	ROC-AUC	PR-AUC	Train ROC-AUC
Logistic Regression	**0.753**	**0.429**	0.797
Random Forest	0.747	0.409	0.992
Gradient Boosting	0.720	0.372	0.952
Business rule	0.618	0.255	–

Logistic Regression is best, and the tree models overfit: near-perfect on training data, no better on new data. Grid-search tuning did not change the ranking. Adding `retention_offer_sent` raised ROC-AUC to 0.86 for every model, a fake gain, so it stayed out. Removing `age` changed ROC-AUC by only 0.001, so it was dropped for fairness; an engineered `effective_fee` feature added nothing.

6. Results and error analysis

On the test set the final model reaches **ROC-AUC 0.763 and PR-AUC 0.413**, in line with CV (bootstrap 95% interval 0.70–0.82). Missing a churner (false negative) costs more than an unnecessary contact (false positive), so the threshold (0.57, max F1 on out-of-fold training predictions) leans towards recall: **55% of churners found at 35% precision** (base rate 18%). Calling the riskiest 20% reaches churners **2.1×** more often than random; the business rule drops to ROC-AUC 0.57 on test.

Missed churners look loyal (14.3 login days vs 8.1 for found churners), and recall is lower for annual (42%) and two-year (12%) contracts than for monthly (66%). These customers leave for reasons not in the data, so the fix is new features, not a bigger model. The strongest drivers are contract type, monthly fee, login days, feature usage and NPS; these are associations, not causes.

7. Deep Learning and Transfer Learning

A PyTorch network (31 → 32 → 16 → 1, ReLU, 20% dropout, weighted binary cross-entropy, Adam, early stopping) reached ROC-AUC 0.749 vs 0.752 for Logistic Regression on the same folds, winning 2 of 5. Validation loss stopped improving after 9–19 epochs while training loss kept falling: early overfitting. With equal performance on ~1,400 rows of tabular data, the simpler model is preferred. Deep Learning would be worth revisiting with far more data, sequences (daily logins) or text.

Transfer learning reuses a model pretrained on large data. It works for images, text and audio because low-level patterns are shared across tasks. Our columns are company-specific, so it does not apply here; it would with support-ticket text (e.g. fine-tuning a Swedish model such as KB-BERT).

8. Recommendation

I recommend **Logistic Regression** (13 features, threshold 0.57) as a monthly risk ranking. It has the best CV ranking scores, the smallest overfitting gap (0.04) and stable folds; its coefficients show why a customer is flagged, it costs almost nothing to run, and it works on little data. **Limitations:** one snapshot, ~320 churners, weaker on long contracts. **Risks:** it shows who is at risk, not who an offer will save, and behaviour can drift. **Next:** validate on a later snapshot, add months-until-renewal and activity-trend features, and A/B test retention offers.

9. MLOps light

The full pipeline is saved with `joblib`. `model_metadata.json` records the 13-column input schema (value ranges, allowed categories), the threshold, the scores and the library versions (Python 3.13.7, scikit-learn 1.9.1, pandas 3.0.6); `requirements.txt` pins the whole environment. `validate_input()` rejects missing columns, out-of-range values and unknown categories. In production I would monitor feature drift (PSI or KS test), missing-value rates, the share flagged, and recall once true labels arrive 30 days later. Retrain on drift (e.g. PSI > 0.2), a clear drop in recall, product or pricing changes, or quarterly. Still missing: a test on a later snapshot, a scheduled scoring job, a model owner, prediction logging and a GDPR review.

10. Limitations and next steps

The data limits the model more than the algorithm does: one snapshot, a small test set (65 churners), feature timing that could not be verified, and possible proxy bias (e.g. region). Next steps: confirm when offers are sent, test on a later snapshot, add renewal and

activity-trend features, then run an A/B test of offers followed by uplift modelling.

11. Sources and tools

Python, pandas, NumPy, scikit-learn, PyTorch, matplotlib, Jupyter in VS Code, and Git. Documentation: scikit-learn user guide, PyTorch docs, pandas docs, and the course material.

AI assistance: I used Claude (Anthropic) as a tutor: to plan the work, explain concepts, suggest code that I ran and checked, review the notebook, and draft this report. I ran every experiment, checked every number against my notebook, and made the final decisions on model, features and threshold.